indexes using the CONCURRENTLY keyword is recommended for any production database where DML operations are occurring concurrently.

How else can you analyze lock behavior?

Using PgBadger For Lock Analysis

You can again use the PgBadger[22] log analysis tool you saw earlier, except this time using the reports it creates for lock analysis.

Using PgBadger is straightforward. Call the executable, passing a single argument which is the path to your postgresql.log file.

Run this command from your Terminal:

sh/pgbadger_log_analysis.sh

```
pgbadger /path/to/postgresql.log
```

The program crunches the data and produces an HTML output file that can be opened in your browser for analysis, named out.html by default. Run man pgbadger to review the large list of optional arguments.

The data presented by PgBadger is historical data from logged activity. Since it relies on log data, it cannot show you what is currently happening.

One of the features of PgBadger is Lock Analysis. Navigate to the top navigation "Locks" tab and explore the information there.

You may want to set a value for log_lock_waits[23] and observe the effect. PostgreSQL runs a deadlock checking routine after 1 second.

You may wan to monitor for row level locking problems when using SELECT ... FOR UPDATE.

What about locking in Active Record?

Active Record Optimistic Locking

In a concurrency context, locks are acquired either optimistically or pessimistically. Pessimistic locks are what PostgreSQL uses, meaning the locks are taken upfront. PostgreSQL supports high levels of concurrent reads and modifications, even with a pessimistic locking strategy.

22. https://github.com/darold/pgbadger
23. https://postgresqlco.nf/doc/en/param/log_lock_waits/